2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion) | 978-1-6654-1219-3/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICSE-Companion52605.2021.00108

UI-based Flaky Tests Datasets

Alan Romano[1], Zihe Song[2], Sampath Grandhi[2], Wei Yang[2], and Weihang Wang[1]

[1]*University at Buffalo, SUNY* [2]*University of Texas at Dallas*

***Abstract*—This artifact submission contains the dataset used in the paper accepted in the ICSE 2021 Technical Track, "An Empirical Analysis of UI-based Flaky Tests". The dataset contains 235 samples of flaky UI tests composed of 152 samples from web projects and 83 samples from mobile projects.**

Obtaining Artifact The artifact is publicly available on Zenodo at https://doi.org/10.5281/zenodo.4456027 and referenceable through doi:/10.5281/zenodo.4456027. It is stored as a ZIP archive that contains the documentation files and a CSV file of the dataset, dataset.csv

Background Flaky tests are software tests that give non-deterministic results when run with the same code version, making their results unreliable. This unreliability wastes computational resources required to run the tests, and it wastes effort in revealing the potentially real fault in the code. These impacts have led to increased attention in flaky test research to create strategies to detect and mitigate against flaky tests; however, most existing work focuses on traditional tests. Our study analyzes UI flaky tests found in web and mobile projects. In order to investigate UI flaky tests, we collected 235 real-world UI flaky test samples found in popular web and Android mobile projects hosted on GitHub. We investigate these issues to find the typical root causes of these flaky UI tests, the strategies used to manifest the flaky behavior, and the strategies used to fix the test flakiness. The findings made in this work can provide a foundation for the development of detection and prevention techniques for flakiness arising in UI tests.

Sample Collection Methodology The sample collection methodology is fully described in the accompanying paper. Overall, we use the GitHub Search API to find repositories leveraging popular web UI frameworks, such as Angular and React, and download all the commits in each of the repositories. We use the patterns "flak*" and "intermit*" and the word "test" in the commit messages to search which commits are related to flaky tests. We use the UI keywords "ui", "gui", "visual", "window", "button", "display", "click", and "animation" to prioritize the commits in the manual inspection process, but all flaky commit messages are inspected.

For mobile projects, we leverage the GitHub Archive dataset to query all GitHub issues relating to repositories using popular UI testing frameworks. Similarly, we search using the same flaky keywords to identify relevant issues and use the UI keywords to prioritize them in the manual inspection.

Table I presents the top 10 repositories that we identified to contain flaky test samples through our methodology. It lists the number of commits that we manually inspected after filtering with the flaky keywords, the number of verified flaky tests in the project, and the size of the project in lines of code.

TABLE I: Top 10 Projects Containing the Most Flaky Tests.

Project	Inspected Commits	Flaky Tests	LOC
Waterfox	937	23	3,949,098
qutebrowser	124	4	45,313
influxdb	81	12	124,591
angular	69	2	135,253
plotly.js	37	24	760,504
material-components-web	26	5	78,972
components	21	5	34,715
oppia	20	2	132,284
wix-style-react	15	11	19,830
streamlabs-obs	13	1	179,184
material-components-android	358	9	11,682
Focus-android	24	8	30,952
RxBinding	21	6	8,787
Xamarin.Forms	140	5	46,609
FirebaseUI-Android	34	4	4,386
Fenix	20	4	155,50
Detox	38	3	2,254
Components	14	3	34,715
Mapbox-navigation-android	4	2	6,201
Sunflower	3	1	354

Manual inspection and Dataset Field Details We provide a clearer description of the dataset and instructions below: The dataset provided contains verified samples of flaky UI tests within web and Android projects. These flaky UI tests can be used to draw insights for other research papers focusing on flaky UI tests or flaky tests in general. The dataset was inspected according to the following procedure:

1) The page is visited through the URL listed in Column B.
   - For the web issues (rows 2-154), the commit page is visited to read the commit message. If the message mentions an issue, the linked issue page is also inspected and recorded in Column J.
   - For the Android issues (rows 156 - 238), the conversation on the issue page and listed commits are read.
2) The file(s) changed within the commit(s) are inspected to identify the UI tests that are affected by the flaky behavior. These are recorded in Column E.
3) The file(s) containing the UI tests are read to identify the testing environment and framework used. These details are recorded in Column F.
   - If these details are not within the commits or issue conversation, then we visit the project page and list the testing environment and frameworks mentioned.
4) The commit messages and linked issue conversations are read to identify the underlying root cause of the flaky

978-1-6654-1219-3/21/$31.00 ©2021 IEEE
DOI 10.1109/ICSE-Companion52605.2021.00108

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 24,2021 at 12:13:35 UTC from IEEE Xplore. Restrictions apply.

behavior. A brief description of the root cause is recorded in Column C and a temporary category name is marked in Column D.

5) The initial post of the linked issue conversation and commit messages are read to identify what the steps the user provided to manifest the issue. A temporary category description fitting these manifestation details is marked in Column G.
6) The issue conversation is read to identify the last commit made before the issue is closed. We read the issue conversation and inspect the last commit to find the fixing strategy used. We record these details in Column H and mark a temporary category description in Column I.
7) After inspecting all of the samples using Steps 1-6, the temporary root cause categories in Column D are reviewed and discussed among the group to find which categories are more representative of the issues and which categories can be merged together. We finalize the categories and update the Column D to use the final category for each sample. The final categories are as follows:
   - Network Resource Loading - Tests attempt to manipulate data from network that has not fully transferred
   - Resource Rendering - Tests attempt to manipulate a UI element that has not fully rendered to the page
   - Animation Timing issue - Tests make assertions on animation scheduling that can change with system load
   - Platform issue - Tests that are affected by an issue belonging to a specific platform and work correctly on other platforms
   - Layout Difference - Tests make assertions on layouts that can be changed depending on browser/Android version run on
   - DOM Selector Issue - Tests are unable to select a DOM element due to differences in browser API behavior
   - Incorrect Test Runner Interaction - Tests use test runner APIs incorrectly or hit unexpected behaviors
   - Time - Tests are sensitive to timing influenced by system load
   - Incorrect Resource Load Order - Test specify actions to be performed on resource before call to load the resource is made
   - Test Order Dependency - Tests fail to clean up resources and are affected by leftover stale data
   - Randomness - Tests use random data that does not match up with expected input
8) We repeat Step 7 for the manifestation categories in Column G. The final categories are as follows:
   - Specify Problematic Platform - Issue author lists the specific platform that they used to reproduce the flaky behavior
   - Reorder-Prune Test Suite - Issue author reduces the test suite methodically to identify which test(s) need to be run to reproduce the issue
   - Reset Configuration Between Tests - Issue author reports that the environment configuration is reset to the default after each test run to reproduce the issue
   - Provide Code Snippet - Issue author provides a code snippet from the test/code under test to reproduce flaky behavior
   - Force Environment Conditions - Issue author provides instructions to set environment conditions necessary to reproduce the issue
   - Unspecified - Issue author does not provide details to reproduce the issue
9) We repeat Step 7 for the fixing strategy categories in Column I. The final categories are as follows:
   - Add/Increase Delay - The time in the delay mechanism is increased or added to the test/code under test
   - Fix Await Mechanism - The incorrect await mechanism is changed to a working mechanism
   - Fix API Access - The incorrect APIs used are replaced with the correct ones
   - Change Library Version - The library is changed to a version that is unaffected by the flaky behavior
   - Refactor Logic Implementation - The code is refactored to fix a program logic issue
   - Disable Animations - Animations are disabled during test runs
   - Remove Test - The test is removed from the test suite

Study Results The collected descriptions were analyzed in order to categorize the samples with similar root causes, manifestation strategies, and fixing strategies. The categories were refined by the authors over several meetings in order to arrive at the final ones listed in the samples. The full results can be found in the accompanying paper, and the results of the root cause categorization can be seen in Table II.

TABLE II: Summary of Root Cause Categories Found

Root Cause Categories	Root Cause Subcategories	Web	Mobile	Total
Timing Issue	Network Resource Loading	15	4	19
	Resource Rendering	47	14	61
	Animation Timing Issue	17	9	26
Environment	Platform Issue	16	18	34
	Layout Difference	8	1	9
Test Runner API Issue	DOM Selector Issue	13	3	16
	Incorrect Test Runner Interaction	10	14	24
Test Script Logic Issue	Strict Comparison Checks	8	1	9
	Incorrect Resource Load Order	11	11	22
	Stale Data	5	5	10
	Random Data Edge Case	2	3	5
	Total	152	83	235

We presented examples of each root cause, manifestation, and fixing strategy category in the accompanying paper. We also present our findings from our analysis of this dataset in the accepted paper, including the prevalence of timing issue root causes (45.1%), the larger number of layout difference root causes in web platforms (5.3%) over mobile projects (1.2%), and the increase in dependency fixes in mobile projects (21.4%) than in web projects (1.3%).

We hope that by making our dataset available and reusable, it will provide researchers interested in flaky tests with a starting point of verified samples to build on.

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 24,2021 at 12:13:35 UTC from IEEE Xplore. Restrictions apply.